

Como classificar um carro sem aprender nenhuma fórmula, só olhando os vizinhos parecidos?



Por que trocar de dataset agora

O dataset de notas (horas_estudo_semana → aprovado) tem só 1 feature — e o KNN funciona medindo distância entre pontos, o que fica pouco interessante (e pouco visual) com uma dimensão só.

Pior: um dos conceitos centrais do KNN é o cuidado com a escala das features — mas isso não faz sentido quando só existe 1 feature.

O dataset mpg, com weight e horsepower (escalas bem diferentes), resolve os dois problemas de uma vez.

**Mesma categoria economico que já criamos na prática de Regressão Logística —
continuidade, não um dataset do nada.**

A ideia central do KNN

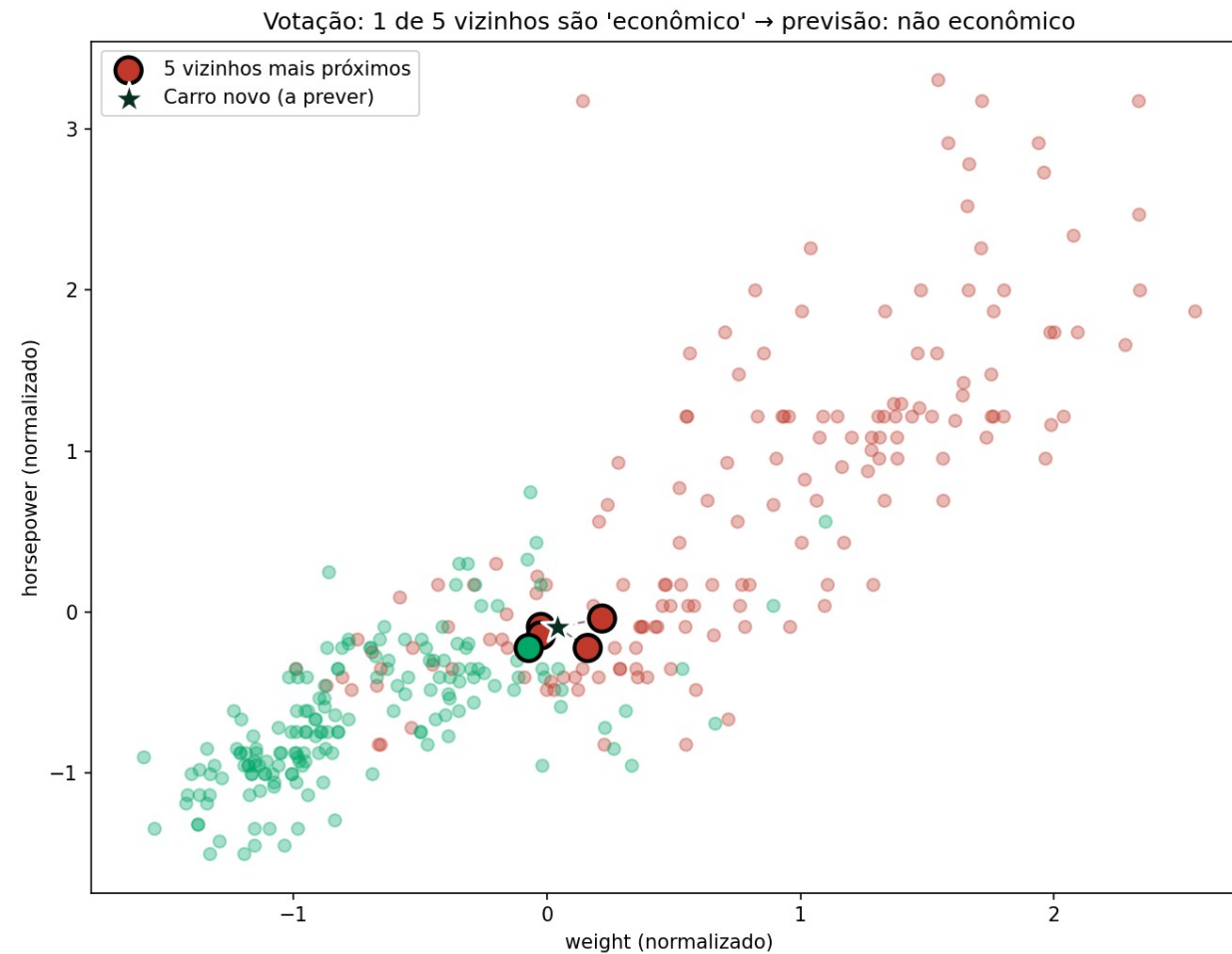
Diferente da regressão logística (que aprende pesos w_1/w_0 e depois esquece os dados originais), o KNN não aprende parâmetro nenhum.

Ele guarda todos os exemplos de treino, e quando chega um caso novo, olha pros K exemplos mais parecidos (mais próximos) e vota pela maioria.

Às vezes chamado de 'aprendizado preguiçoso' (lazy learning) — não há fase de treino de verdade, só de consulta.



A votação, visualmente



Por que a escala das features importa

KNN mede distância entre pontos — se uma feature tem escala muito maior que outra (weight, na casa dos milhares, vs. horsepower, dezenas), ela sozinha domina a distância.

Isso distorce quem é considerado 'vizinho próximo', mesmo que as duas features sejam igualmente relevantes.

A solução é a mesma que já vimos: StandardScaler, deixando tudo na mesma escala.

Vamos ver na prática

```
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors

X = df[["weight", "horsepower"]].values
carro_novo = np.array([[3000, 100]])

nn = NearestNeighbors(n_neighbors=5).fit(X)
_, idx_sem = nn.kneighbors(carro_novo)

scaler = StandardScaler().fit(X)
X_norm = scaler.transform(X)
carro_novo_norm = scaler.transform(carro_novo)
nn2 = NearestNeighbors(n_neighbors=5).fit(X_norm)
_, idx_com = nn2.kneighbors(carro_novo_norm)

print("Sao os mesmos?", set(idx_sem[0]) == set(idx_com[0]))
```



O papel do K

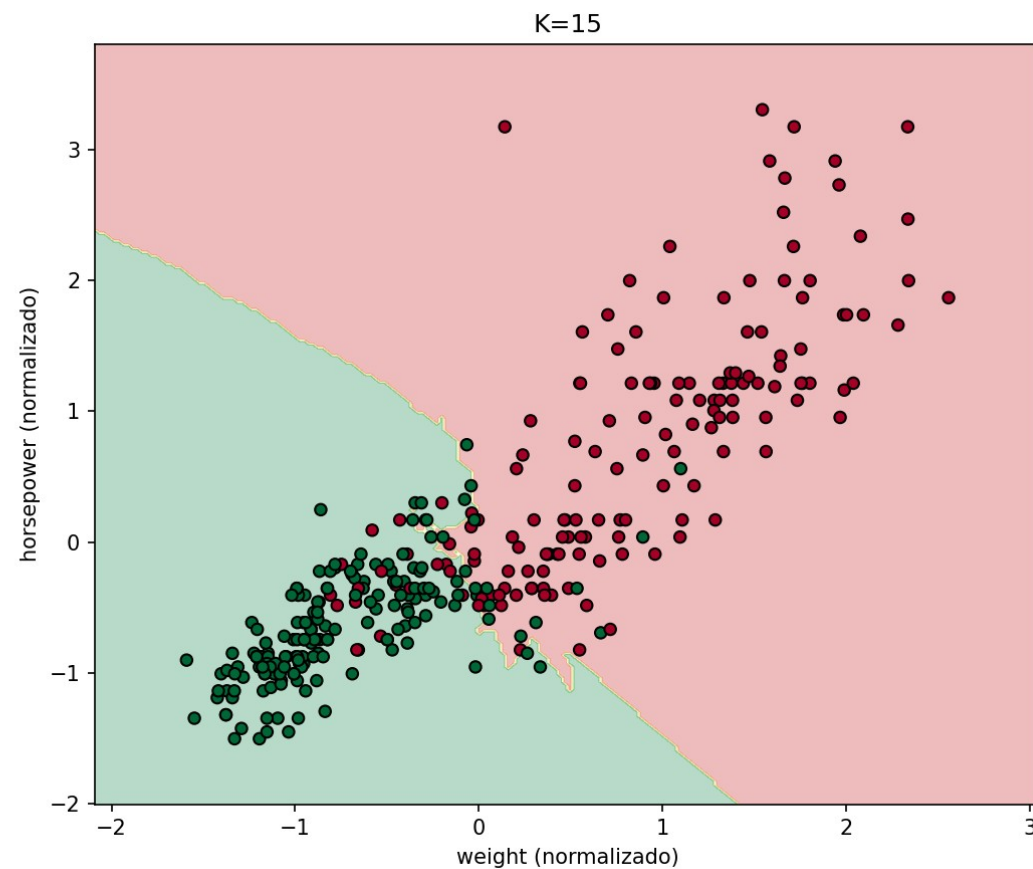
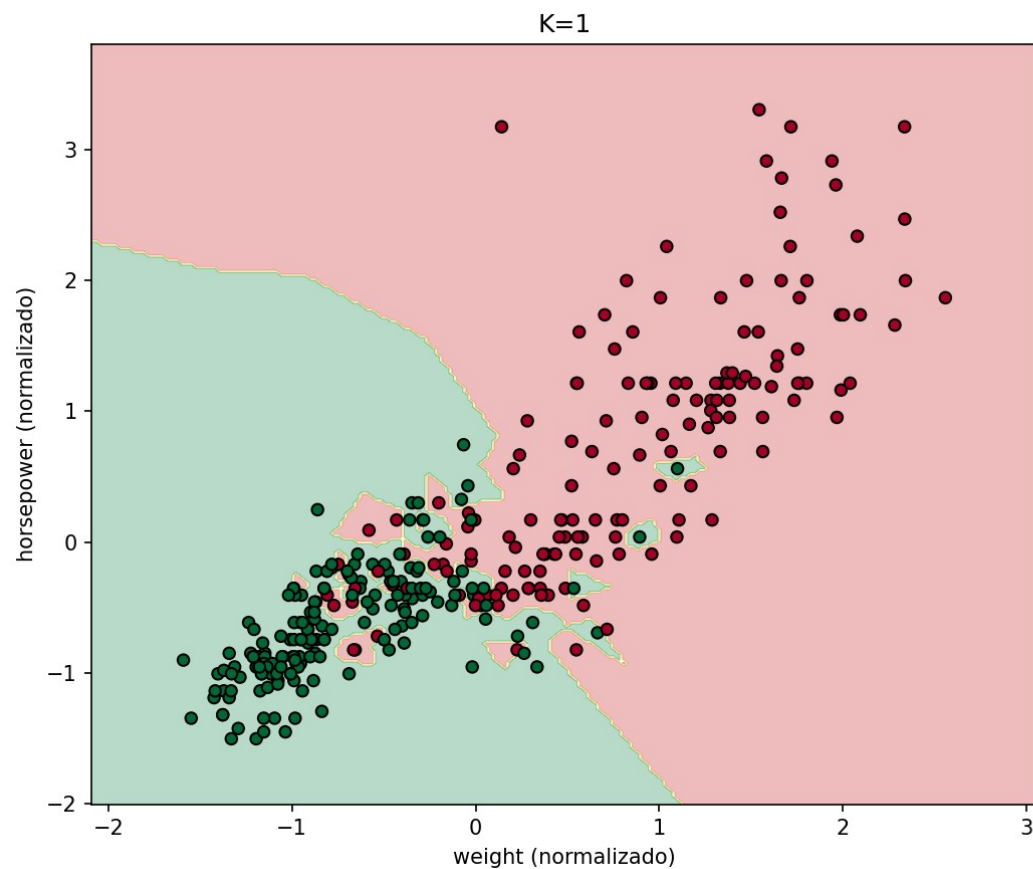
K muito pequeno (tipo 1 ou 3) deixa o modelo muito sensível a ruído — um único vizinho estranho pode mudar a decisão.

K grande demais suaviza excessivamente, ignorando padrões locais legítimos.

Equilíbrio parecido com o overfitting/underfitting que já vimos na regressão polinomial.



K pequeno vs. K grande



O fluxo no Scikit-Learn

Mesma gramática de sempre: `KNeighborsClassifier(n_neighbors=K)`, `fit()`, `predict()` — praticamente idêntico ao que já fizeram com a regressão logística.

A diferença: aqui, normalizar ANTES de treinar não é opcional — é parte obrigatória do fluxo.

`fit_transform()` no treino, `transform()` no teste — mesma regra de ouro de sempre.



Vamos ver na prática

```
from sklearn.model_selection import train_test_split

X = df[["weight", "horsepower"]]
y = df["economico"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=5, stratify=y
)

scaler = StandardScaler()
X_train_norm = scaler.fit_transform(X_train)
X_test_norm = scaler.transform(X_test)
```



Vamos ver na prática

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

modelo = KNeighborsClassifier(n_neighbors=5)
modelo.fit(X_train_norm, y_train)

pred = modelo.predict(X_test_norm)
print(f"Acuracia com K=5: {accuracy_score(y_test, pred):.2%}")
```



Como escolher o melhor K

Não tem fórmula mágica — o comum é testar vários valores de K e comparar a acurácia de cada um no conjunto de teste.

Um 'grid search' simples, feito na mão com um for.



Vamos ver na prática

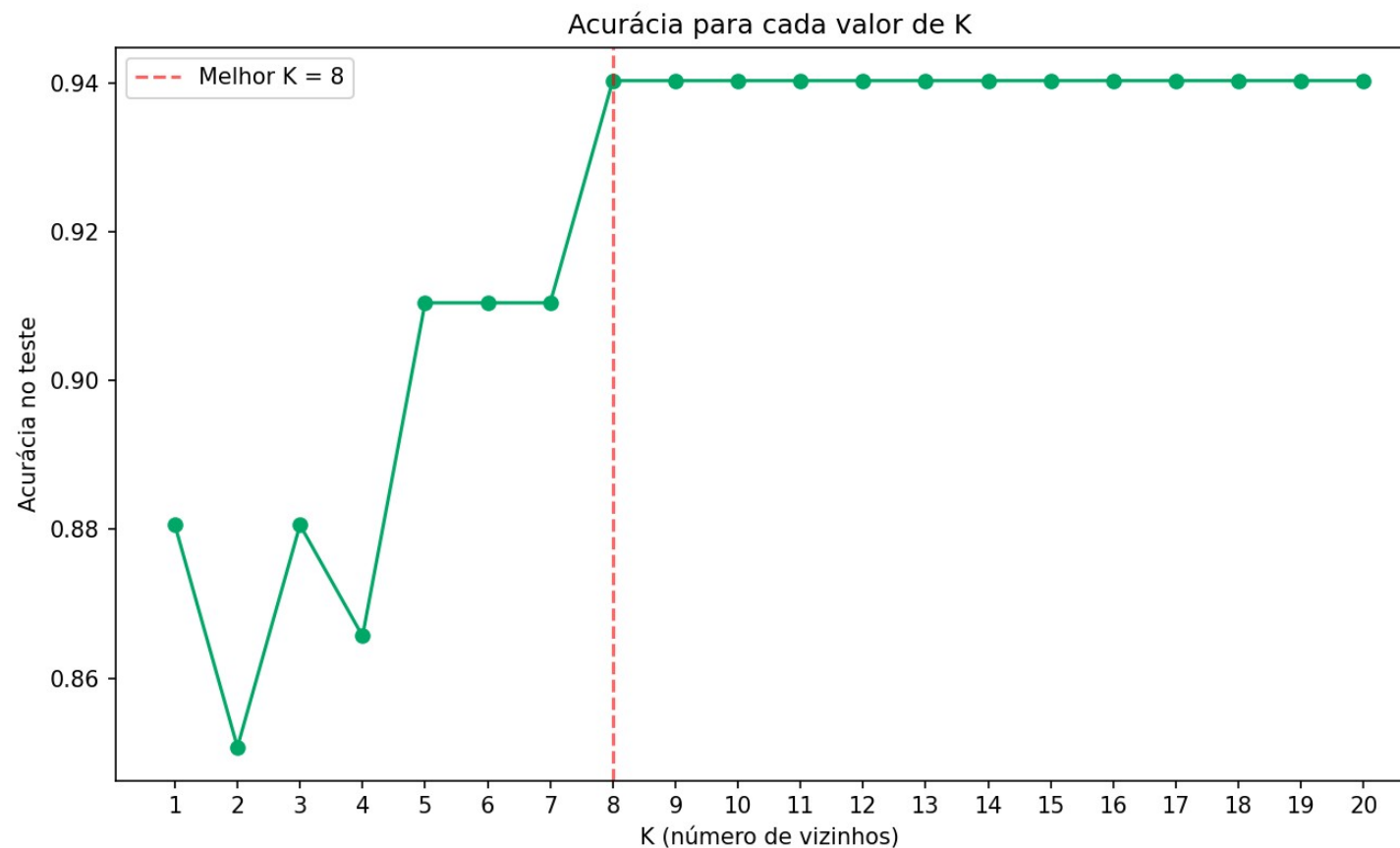
```
valores_k = range(1, 21)
acuracias = []
for k in valores_k:
    m = KNeighborsClassifier(n_neighbors=k).fit(X_train_norm, y_train)
    acuracias.append(accuracy_score(y_test, m.predict(X_test_norm)))

melhor_k = list(valores_k)[np.argmax(acuracias)]
print(f"Melhor K: {melhor_k}")

plt.plot(list(valores_k), acuracias, marker="o")
plt.axvline(melhor_k, color="red", linestyle="--")
plt.show()
```



Encontrando o melhor K



Decidir por semelhança, sem aprender fórmula nenhuma

- 1 **Votação por vizinhos** — Sem parâmetro aprendido — só consulta aos K mais próximos
- 2 **Escala importa muito** — StandardScaler deixou de ser opcional
- 3 **K é o equilíbrio** — Pequeno demais = ruído; grande demais = suaviza demais

Fim da Sessão 1 — Regressão Logística e KNN. Sessão 2: Árvores, Random Forest e Avaliação.

